

labwork 5

October 2, 2026

0.1 Gaussian blur with CPU

I first tried to make a Gaussian blur using the CPU to make it easier for me to understand it. The goal is for each pixels, to make a weighted average of its neighbors, take the result and make it its new value.

To do that we start by iterating on each pixels. We start and stop 3 pixels before the edge, this means we live a border of 3 unchanged pixels all around the image.

Then we iterate again on every pixels of the grid (7x7) and make the weighted average.

```
def gaussianBlurCPU(src , gaussianGrid):
    height = len(src)
    width = len(src[0])
    result = np.zeros((height , width , 3), np.uint8)
    for i in range(3, height - 3):
        for j in range(3, height - 3):
            tot_r = tot_g = tot_b = 0
            for v in range(-3, 4):
                for w in range(-3, 4):
                    tot_r += gaussianGrid[v + 3][w + 3]*int(src[i + v][j + w][0])
                    tot_g += gaussianGrid[v + 3][w + 3]*int(src[i + v][j + w][1])
                    tot_b += gaussianGrid[v + 3][w + 3]*int(src[i + v][j + w][2])

            result[i][j][0] = tot_r/1003
            result[i][j][1] = tot_g/1003
            result[i][j][2] = tot_b/1003

    return result
```

0.2 Gaussian blur with GPU

The second step is to do the same but with the GPU. This is pretty much the same, we only adapt it to the kernel. We want to have as much threads as pixels, because round the block size to the superior we need to check that the thread is corresponding to a pixel, if not we stop.

The rest of the function is exactly the same as the CPU.

```
@cuda.jit
def gaussianBlur(src, dst, grid):
    x = cuda.threadIdx.x + cuda.blockIdx.x * cuda.blockDim.x
    y = cuda.threadIdx.y + cuda.blockIdx.y * cuda.blockDim.y
    if y > src.shape[0] or x > src.shape[1]:
        return
    tot_r = tot_g = tot_b = 0
    for v in range(-3, 4):
        for w in range(-3, 4):
            tot_r += grid[v + 3][w + 3]*int(src[y + v][x + w][0])
            tot_g += grid[v + 3][w + 3]*int(src[y + v][x + w][1])
            tot_b += grid[v + 3][w + 3]*int(src[y + v][x + w][2])

    dst[y, x, 0] = tot_r/1003
    dst[y, x, 1] = tot_g/1003
    dst[y, x, 2] = tot_b/1003
```

0.3 Gaussian blur with GPU optimized with shared memory

The last step is to use the shared memory to optimize the GPU kernel function.

First, we define the values that will be loaded on shared memory. Here, we define 3 array of 16 by 16 containing 8 bits integers. Each arrays contain every value of a color in the block. The block size (16 here) is hard coded because this value cannot change, even between executions.

```
shared_r = cuda.shared.array((16, 16), np.uint8)
shared_g = cuda.shared.array((16, 16), np.uint8)
shared_b = cuda.shared.array((16, 16), np.uint8)

shared_r[yInBlock, xInBlock] = src[y, x, 0]
shared_g[yInBlock, xInBlock] = src[y, x, 1]
shared_b[yInBlock, xInBlock] = src[y, x, 2]
```

Before trying to access this value we need to wait for every threads to be ready.

```
cuda.syncthreads()
```

Then, we use the same method as before except this time we access the shared memory instead of the global memory.

```
tot_r += grid[v + 3][w + 3]*int(shared_r[yInBlock + v][xInBlock + w])
tot_g += grid[v + 3][w + 3]*int(shared_g[yInBlock + v][xInBlock + w])
tot_b += grid[v + 3][w + 3]*int(shared_b[yInBlock + v][xInBlock + w])
```

0.4 Blocksize benchmark

Finally, we want to check if our kernel with shared memory is faster or not. We also want to see for which block size our kernels are faster. The problem is that the block size value cannot be change during the execution so we need to collect the values of the benchmark by hand.



